REVERSE ENGINEERING · RED/GREEN VALIDATION · REAL WINDOWS CLIENT

JFTSE Tournament Mode

A first-principles, client-validated implementation of the tournament list, detail, registration, cancellation, and 16-entry final bracket protocol.

> **Result:** the unmodified client renders the complete recovered vertical slice against local auth and game servers, including accepted recurring bracket match-state polling.

Branch	`feature/tournament-mode`
Development base	`65c36651`
Faulty commit removed	`fcdeb89a94bb17337f8b8f499b79c5892370004e`
Revert commit	`e817af5a`
Implementation commit	`75e61191`
Validation date	11 August 2026

Contents

Executive result

This work restores a client-validated Tournament Mode vertical slice that did not exist on the development branch:

- the real client lists a scheduled `JFTSE Open Cup`;
- tournament metadata, dates, mode, entry type, and winner rewards render;
- a player can apply, see the applied state, cancel, and see the nonparticipant state restored;
- the enabled final-tournament-bracket control opens a real bracket screen;
- the client renders a 16-entry final bracket and continuously polls its 15 internal match records;
- the bracket zoom, round-reward panel, close, and back controls work without disconnecting the client;
- all recovered request/response packets are generated, registered, handled, and covered by byte-level tests;
- fixed-cardinality schema arrays now serialize without an erroneous count prefix.

This is not represented as a completed production tournament engine. The wiki describes the original broader mode as a 64-player tournament. The client evidence recovered here is specifically the 16-entry **final-stage bracket** selected by `bracketType = 1`. Qualifying, tournament match orchestration, archives, prize settlement, durable scheduling, and database persistence remain deliberately unimplemented because their contracts were not observed. Disabled client controls were not made to send invented packets.

No client binary patch was required. The downloaded archive was preserved, while a runtime copy was configured for the loopback services and instrumented only during protocol diagnosis.

Documentation used as the game specification

The JFTSE wiki was treated as first-party project documentation:

1. JFTSE Roadmap explicitly lists Tournament Mode under both reverse engineering and planned emulator work. It also warns that packet identification alone is insufficient: surrounding structures and value effects require trial and error.
2. Game Modes describes registration into available tournaments and a 64-player bracket in the original overall mode.
3. Packet Structure defines the fixed eight-byte header, little-endian fields, packet ID, and payload length.
4. Packet Schema (.packet) Format defines generated CMSG parsing, server packet serialization, UTF-16LE strings, Windows FILETIME dates, repeated fields, and `[len = N]` fixed arrays.
5. Database Schema & Cheatsheet documents the existing schema and configuration model. It does not document tournament tables. No speculative migration was added.
6. All Pages was reviewed to identify all relevant protocol, schema, mode, and roadmap material.

The wiki establishes the product intent and common wire conventions. Exact Tournament Mode bytes were derived from the client, not guessed from prose.

Scope boundary

Capability	Result	Evidence basis
Tournament list	Implemented	Real-client request/response and visible row
Detail and rewards	Implemented	Real-client `0x26AD/AE` and `0x26BE/BF` exchange
Apply	Implemented in memory	Confirmation, success modal, applied state
Cancel	Implemented in memory	Confirmation, success modal, restored state
Final bracket	Implemented	Visible 16-entry bracket, parser `c3_ok` trace
Match-state polling	Implemented	Repeated `0x26C2/C3` every approximately 10 seconds
Bracket zoom/reward/back UI	Validated	Real-client screenshots
Qualifying bracket	Not implemented	Client control disabled; no observed request contract
Tournament game start	Not implemented	Client control disabled; no complete lifecycle contract
Archives	Not implemented	Client control disabled; no observed request contract
64-player qualifying stage	Not implemented	Wiki-level requirement only; no validated wire structure
Durable database state	Not implemented	No documented tournament schema; current slice is deterministic/in-memory
Prize settlement	Not implemented	Display contract recovered; award transaction contract not recovered

Branch and faulty commit removal

The requested faulty commit was removed with a dedicated revert:

```
e817af5a Revert "fix types in pet packets; adjust S2CRoomPlayerListInformationPacket to include pet;
room player cant change slots with pet"

This reverts commit fcdeb89a94bb17337f8b8f499b79c5892370004e.
```

After implementation, the latest development branch was merged. Development contained a follow-up that depended on the faulty BattleMon room-packet shape; the merge conflicts were resolved in favor of the explicit revert, so the removed payload was not silently reintroduced. The latest unrelated development fixes are otherwise present.

First-principles reverse-engineering method

The implementation followed a strict evidence loop:

All client/server tests used loopback addresses. No public game-server endpoint was configured.

Client identity

```
FantaTennis.7z
SHA-256 c19ca21b8e2ab091953b2f631e48853b6477400f4d7000682ac7440f9994f12e

FantaTennis.exe
SHA-256 5477f0827acae66976403aecd2e9ebffeb4fa28da1fedae5f9541ec25e336c31
Size     3,801,088 bytes
```

Recovered packet family

Every field below is little-endian after the standard eight-byte packet header.

ID	Direction	Purpose	Recovered payload
`0x26AD`	C→S	Open/join tournament detail	`tournamentId:int32`
`0x26AE`	S→C	Detail metadata	`status:int8` ; on success, metadata through bracket size
`0x26AF`	C→S	List page	`page:int32`
`0x26B0`	S→C	Tournament list	count-prefixed tournaments
`0x26B1`	C→S	Apply	`tournamentId:int32`
`0x26B2`	S→C	Apply result	`status:int8`
`0x26B3`	C→S	Cancel	`tournamentId:int32`
`0x26B4`	S→C	Cancel result	`status:int8`
`0x26BE`	C→S	Participation/detail state	`tournamentId:int32`
`0x26BF`	S→C	Participation/detail state	result plus success-only player state block
`0x26C0`	C→S	Bracket definition	`tournamentId:int32, bracketType:uint8, page:uint8`
`0x26C1`	S→C	Bracket definition	status plus selectors and fixed-width entry rows
`0x26C2`	C→S	Bracket match-state poll	`tournamentId:int32, bracketType:uint8, matchIndex:uint8`
`0x26C3`	S→C	Bracket match-state result	status plus selectors, `count:int16` , two-byte records

Only final selector `(bracketType = 1, page/matchIndex = 0)` is accepted. Unsupported selectors receive a terminal nonzero response instead of final-stage bytes disguised as another structure.

`0x26B0` fixed tournament arrays

The list structure contains:

- exactly five reward records, each two `int32` values;
- exactly 16 pairs in bracket array A;
- exactly 16 pairs in bracket array B.

The client consumes these arrays directly, without a count before each array. The pre-existing generator wrote a count for every repeated server field, even when the schema declared `[len = N]`. That shifted every subsequent byte and prevented a valid tournament record.

`FTPacketGen` now treats fixed repeated fields symmetrically:

1. no count is written on the wire;
2. null or wrong-size values fail fast;
3. primitive and composite repeated fields follow the same rule;
4. normal repeated fields retain their old count-prefixed behavior.

The golden tests assert both the complete tournament byte order and wrong-cardinality failures.

`0x26C1` final bracket definition

Accepted success payload:

```
status          int8       0
tournamentId    int32      1
bracketType     uint8      1 (final)
page            uint8      0
unknown         uint8      0
entryCount      int16      16
entries[16]:
  first         12 bytes   fixed UTF-16LE, null-padded
  second        12 bytes   fixed UTF-16LE, null-padded
  third         12 bytes   fixed UTF-16LE, null-padded
```

Payload size:

```
1 + 4 + 1 + 1 + 1 + 2 + (16 × 36) = 586 bytes
586-byte payload + 8-byte header = 594-byte TCP packet
```

`0x26C3` match records and the decisive sentinel

The client derives its structural expectations from the final-bracket metadata:

```
entries  = 16
rounds   = 5
nodes    = 31
matches  = nodes - entries = 15
```

The response layout is:

```
status          int8       0
tournamentId    int32      1
bracketType     uint8      1
matchIndex      uint8      0
matchCount      int16      15
matches[15]:
  result        int8       -1 (0xFF, empty/unresolved sentinel)
  state         uint8      0
```

Payload size:

```
1 + 4 + 1 + 1 + 2 + (15 × 2) = 39 bytes
39-byte payload + 8-byte header = 47-byte TCP packet
```

Sending 15 records of `00 00` passed the count comparison but failed a later client state validator. Runtime uprobes narrowed the rejection to the branch after the count check. Static analysis of that branch identified a signed sentinel domain; changing only the first state byte to `0xFF` produced the client success branch and visible bracket.

The preserved success trace states:

```
c3_count:   loaded=1 complete=0 entries=16 rounds=5 nodes=31 expected=15
c3_compare: got=0xf expected=0xf
c3_ok:      loaded=1 complete=0 expected=15
```

Subsequent polls show `complete=1` and continue through `c3_ok`.

Server implementation

Packet schema

`server-core/src/main/packets/tournament/Tournament.packet` declares the recovered packet IDs, request fields, shared tournament records, and fixed arrays.

Handlers

`game-server/src/main/java/com/jftse/emulator/server/core/handler/tournament/` contains one handler per recovered client request:

- list and join/detail metadata;
- apply and cancel;
- participation information;
- final-bracket definition;
- final-bracket match-state polling.

Success-only tails are written manually where the response is a protocol union: a nonzero first byte terminates the packet, while zero requires a larger body. Modeling only the discriminant in `.packet` avoids auto-writing fields that must not exist on failure.

State model

`TournamentManager` owns the smallest deterministic state needed to validate the recovered flow:

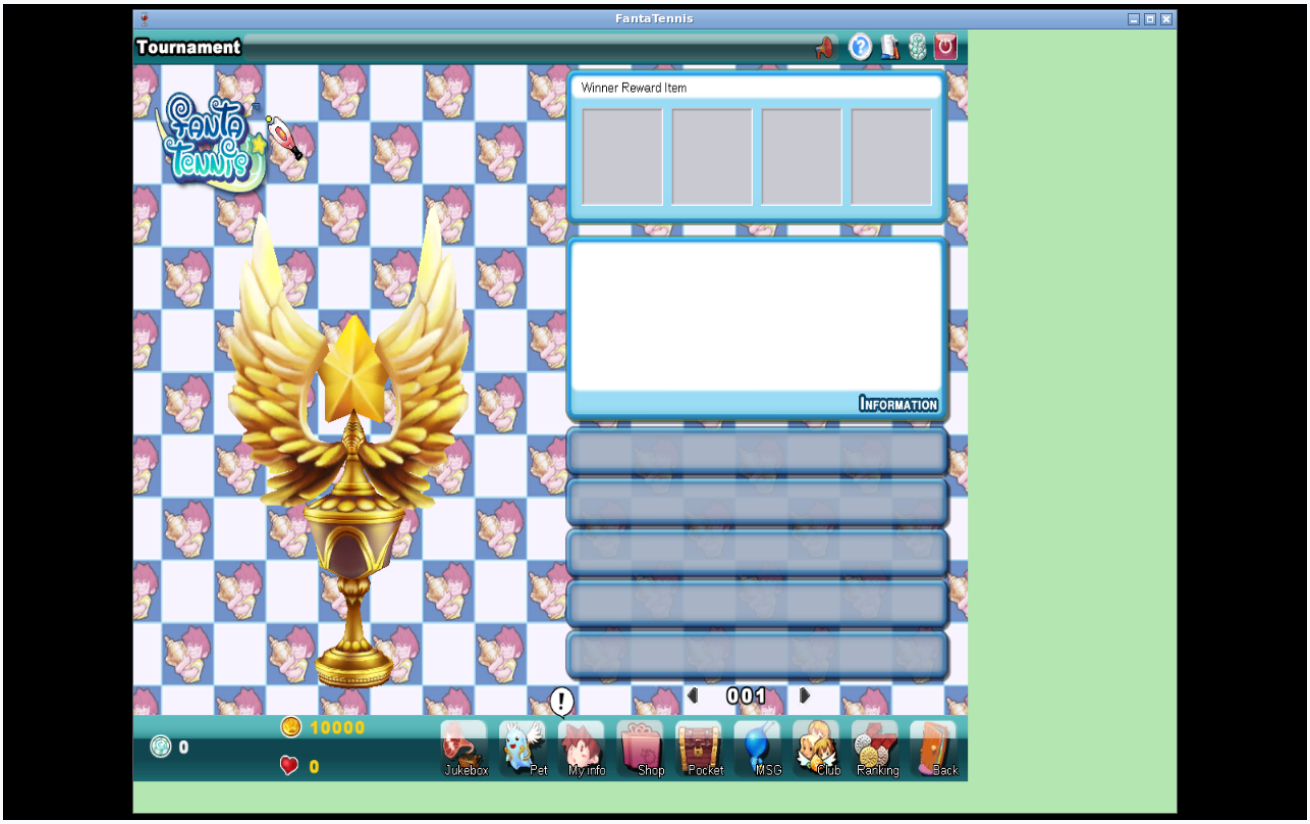
- one scheduled `JFTSE Open Cup`;
- UTC dates relative to server start;
- a concurrent per-tournament player-ID application set;
- deterministic bracket labels;
- 15 immutable unresolved match records.

Application state is isolated per player and safe for concurrent apply/cancel operations. Restarting the game server clears it. This is intentional for the recovered slice: adding entities or migrations before the tournament schema and lifecycle are known would convert guesses into long-lived compatibility obligations.

Red/green development record

Red 1: no tournament handlers

The development server accepted the client connection but did not answer the list request. The tournament screen contained no row.

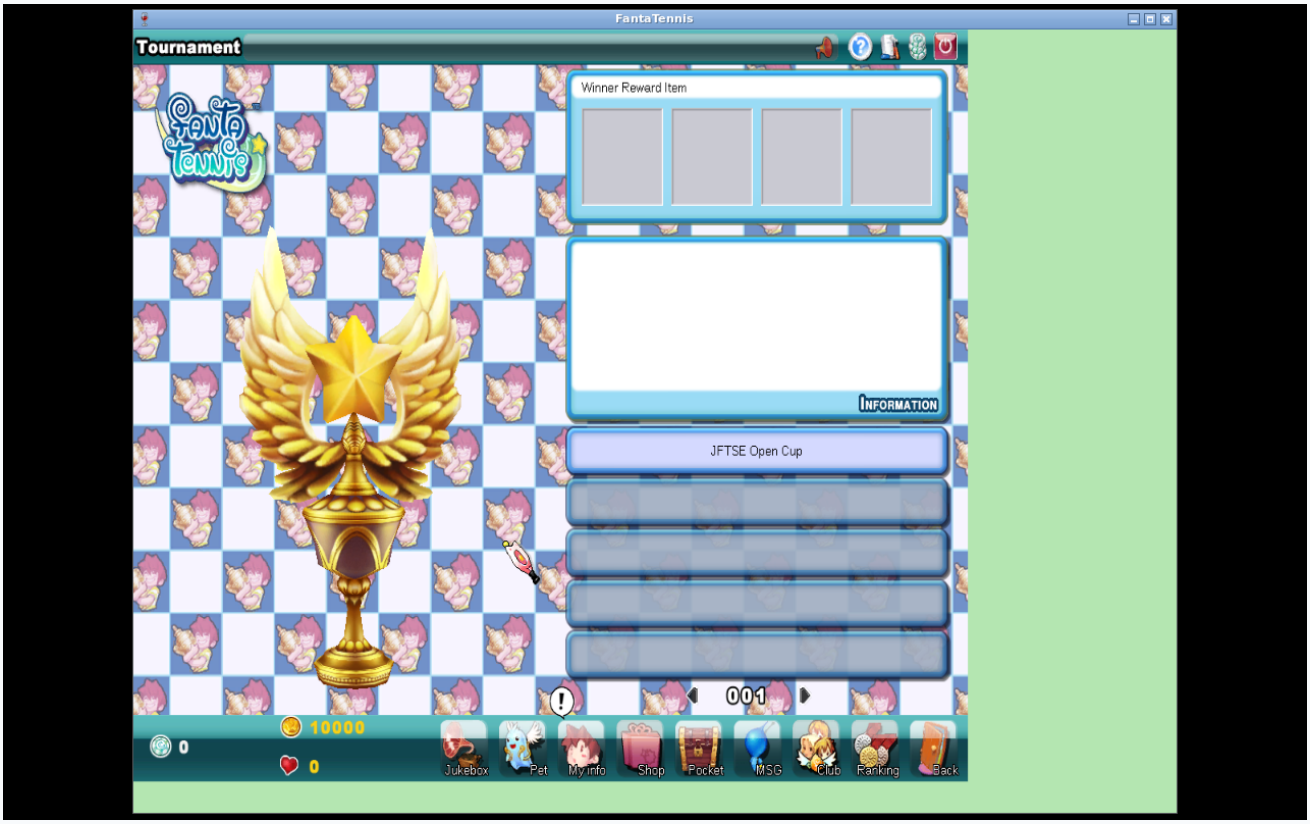


Artifacts:

- `client-server-baseline.pcap`
- `game-server-no-tournament-handler.log`
- `maven-tournament-tests-before-implementation.log`

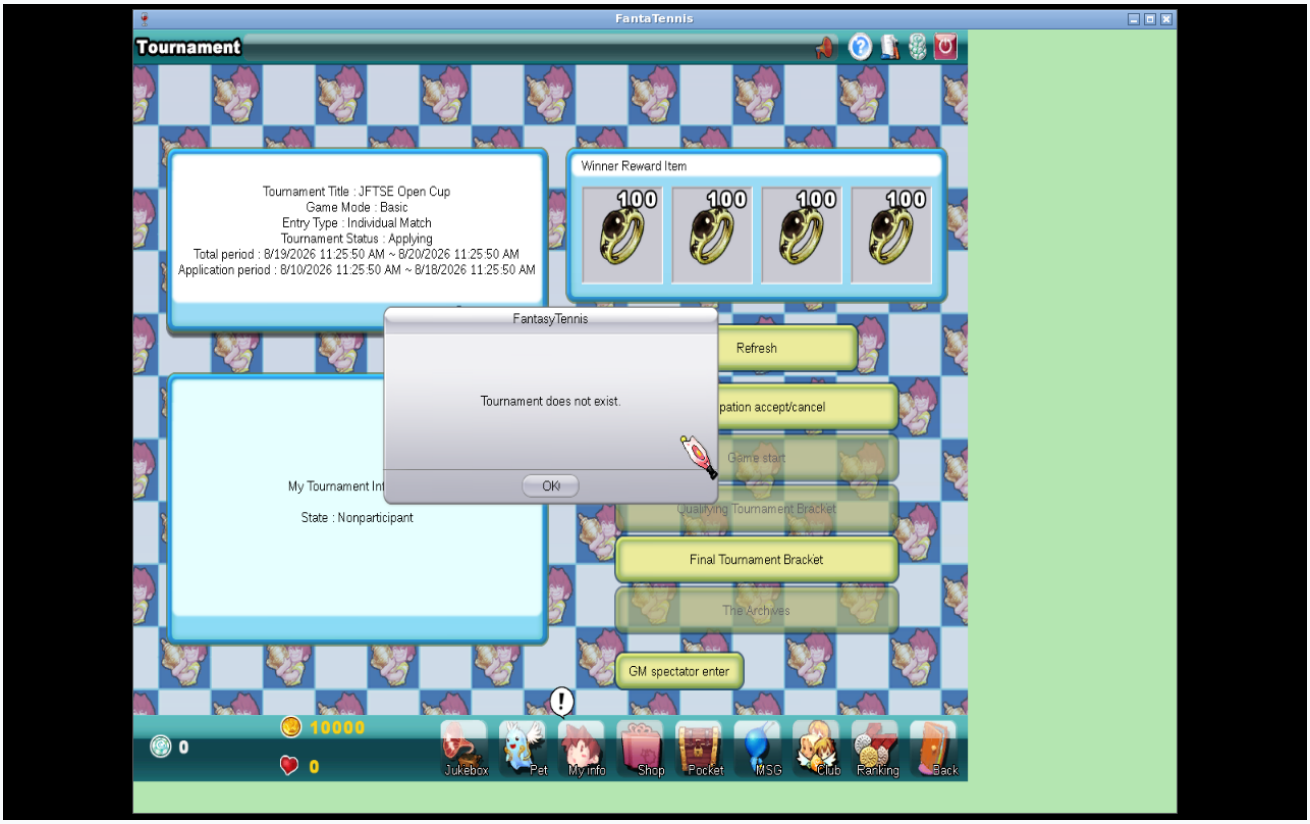
Red 2: list existed, detail response was missing

Selecting the row produced `0x26AD` and `0x26BE`, but the screen could not populate correctly without both metadata and player information responses.



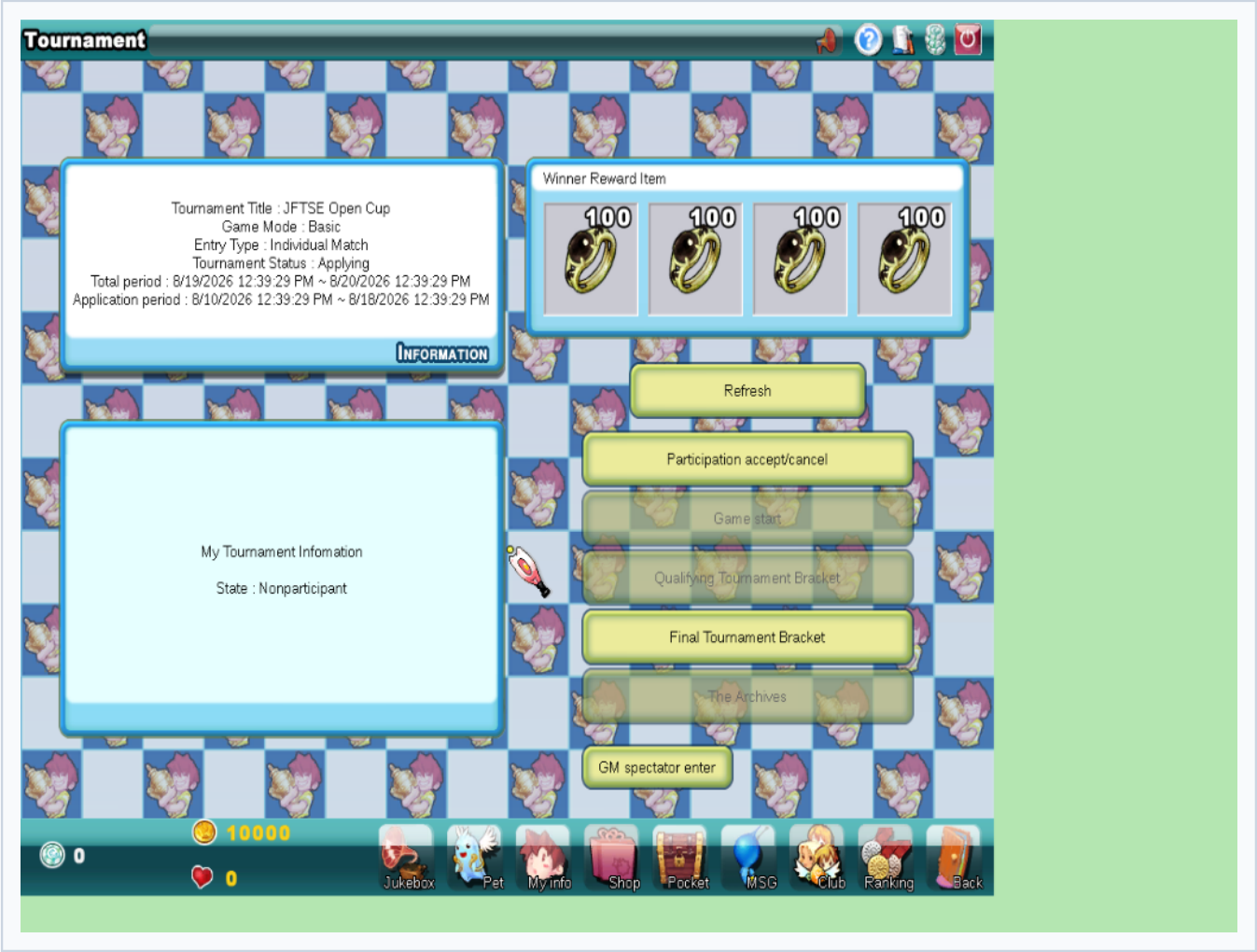
Red 3: `0x26AE` omitted mandatory success metadata

Returning only a success byte was not sufficient. The client needed the tournament metadata block after the successful status.



Red 4: final bracket request unhandled

Clicking the enabled final bracket button revealed `0x26C0` with a six-byte payload.

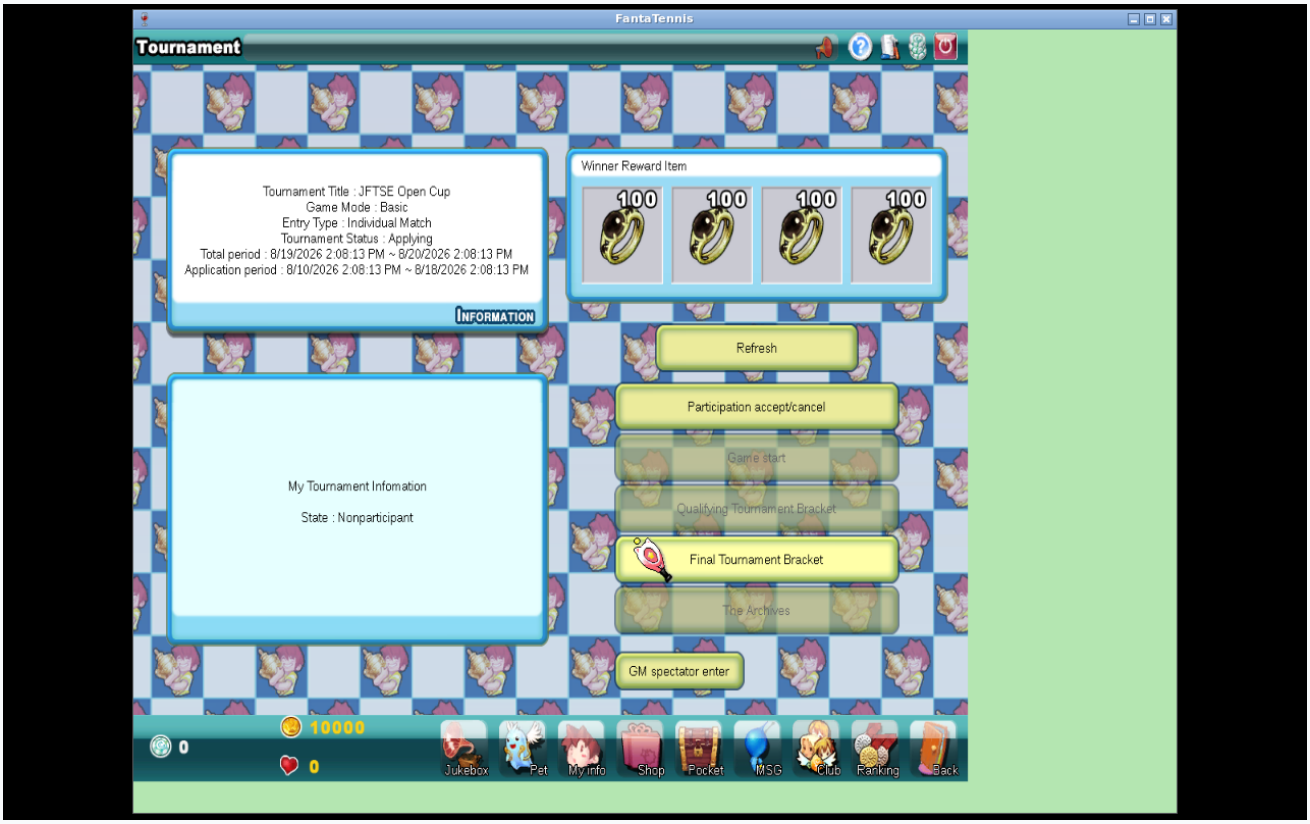


Red 5: definition accepted, automatic match request unhandled

A structurally accepted `0x26C1` caused the client to send `0x26C2` automatically. Without `0x26C3`, the detail screen remained visible.

Red 6: correct count, invalid record state

Fifteen `00 00` records satisfied `got=15, expected=15`, but the parser still took its failure return. This was the critical proof that cardinality alone was insufficient.



Artifacts:

- `bracket-zero-state-rejected.pcap`
- `client-c3-parser-trace.txt`
- `game-server-bracket-complete.log`
- `maven-tournament-bracket-match-before-implementation.log`

Green: exact packet contract and real-client flow

Focused final test result:

```
Tests run: 21, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

Full Java 21 reactor result after merging current development:

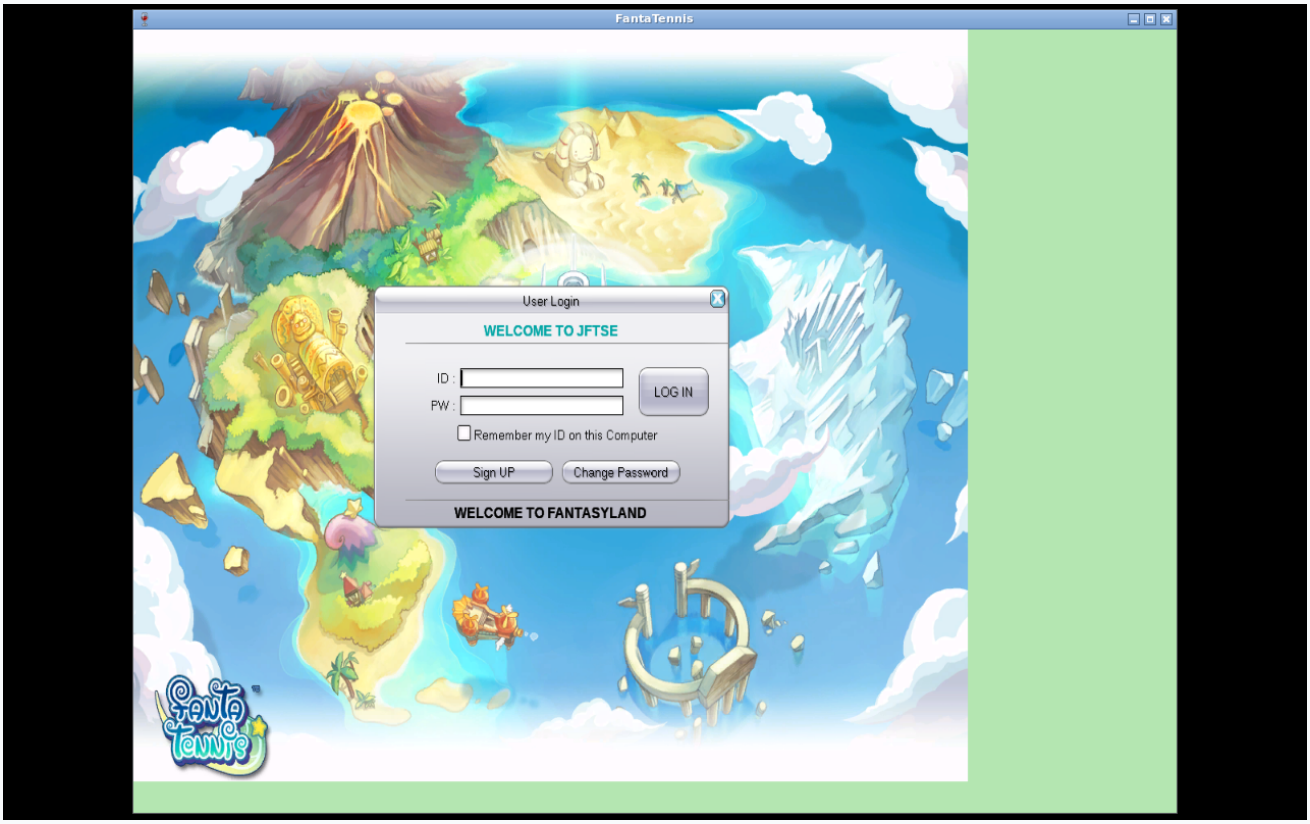
```
Tests run in game-server: 28, Failures: 0, Errors: 0, Skipped: 0
All 11 reactor modules: SUCCESS
BUILD SUCCESS
```

Artifacts:

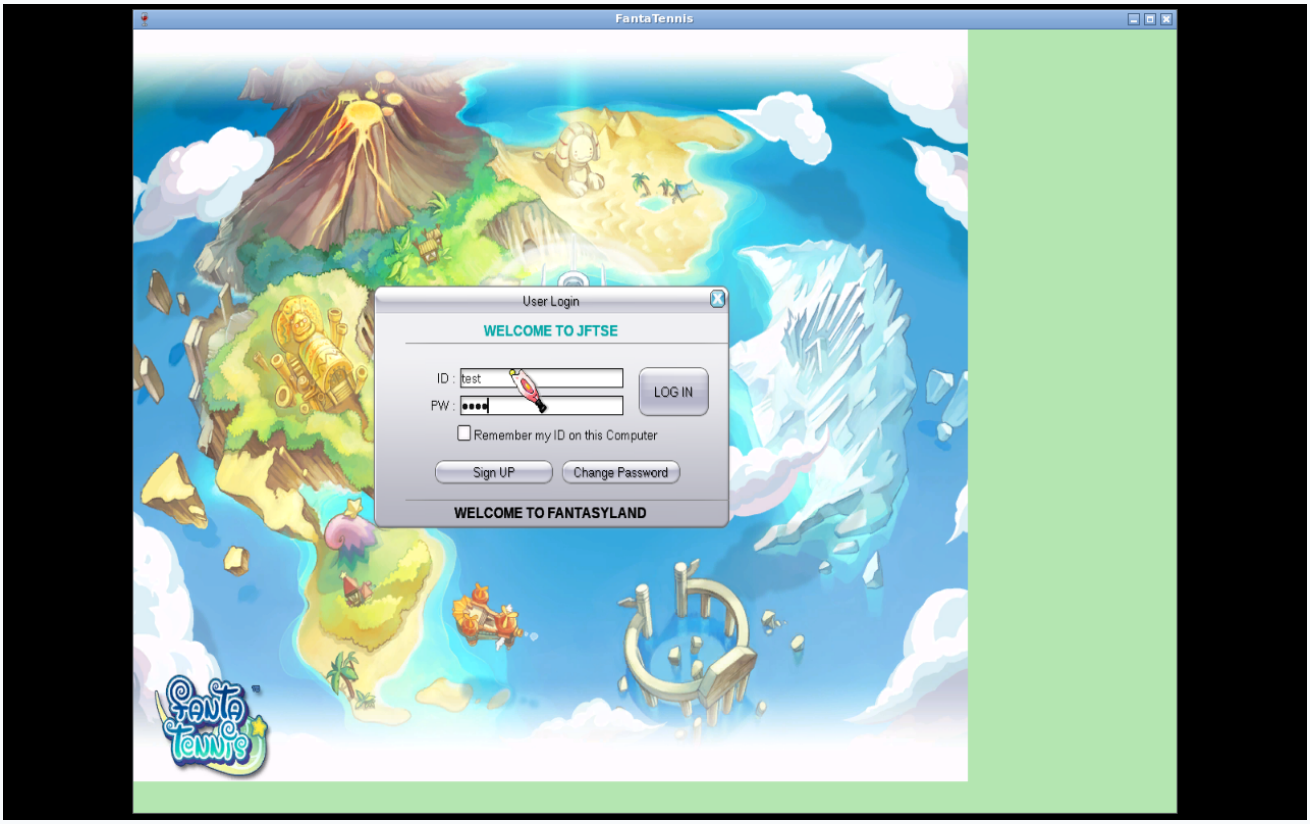
- `maven-tournament-final-review-fixes.log`
- `maven-full-reactor-tests-final.log`

Real-client end-to-end visual evidence

1. Client reached the local authentication server

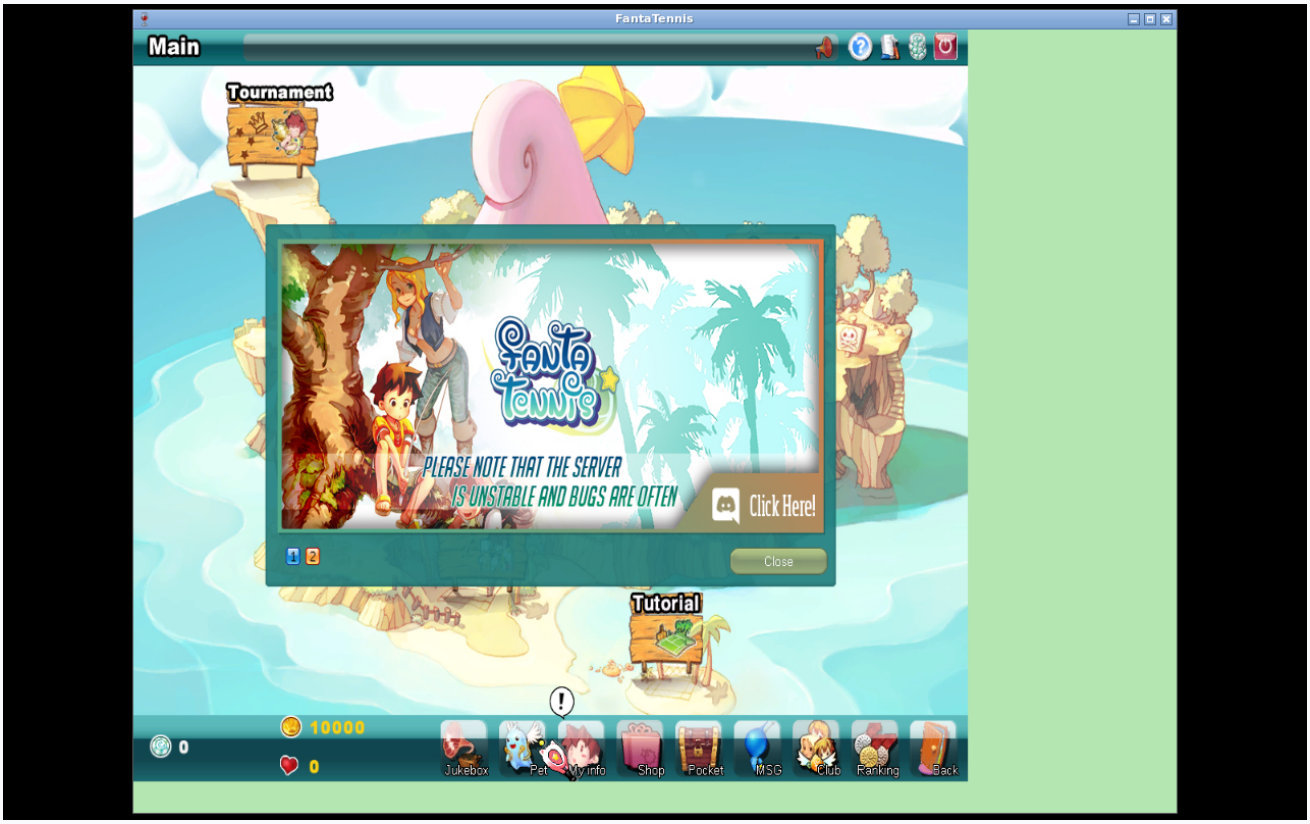


2. Test credentials were entered



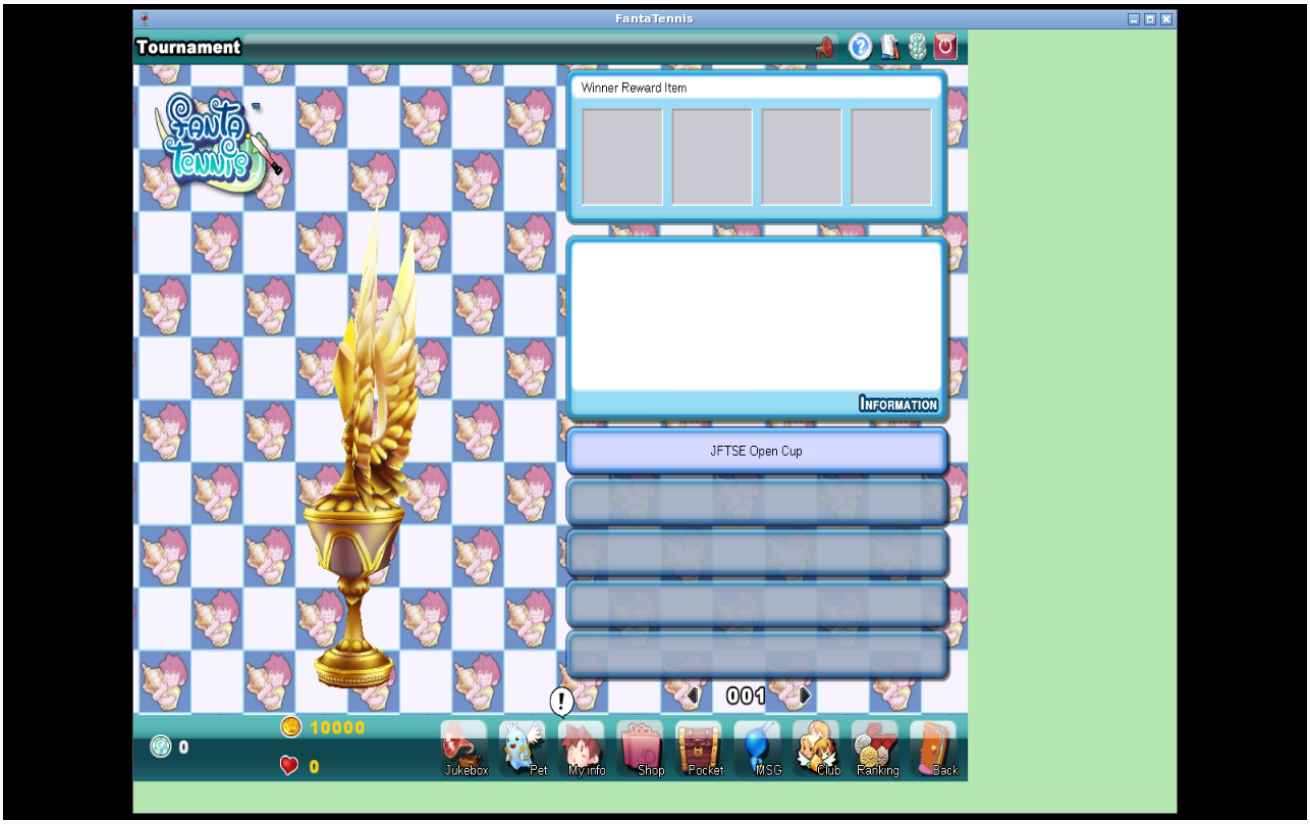
3. Test character and local channel were selected

4. Client connected to the local game server



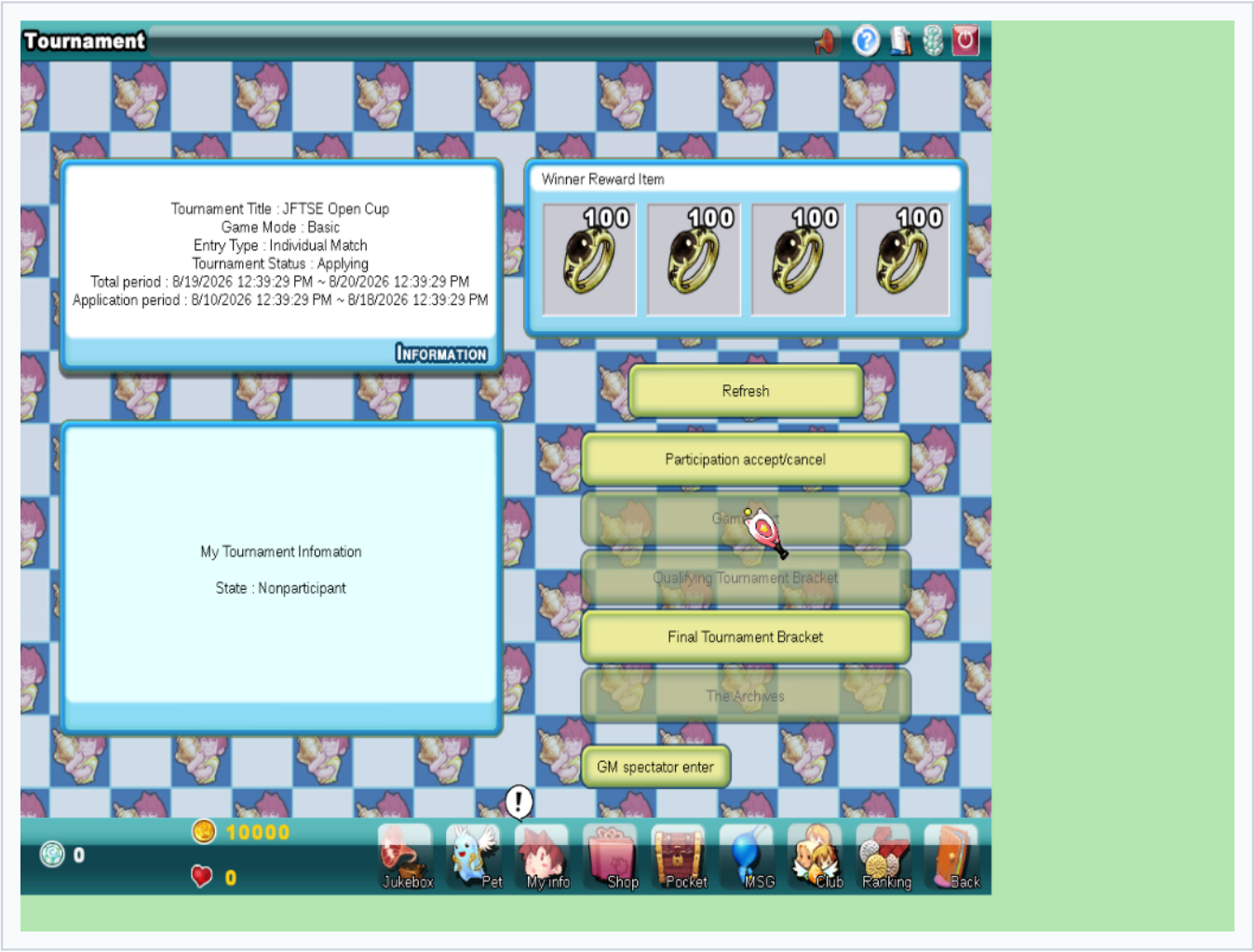
5. Tournament list rendered

The deterministic tournament appears as `JFTSE Open Cup`; the client also renders its page control and tournament art.

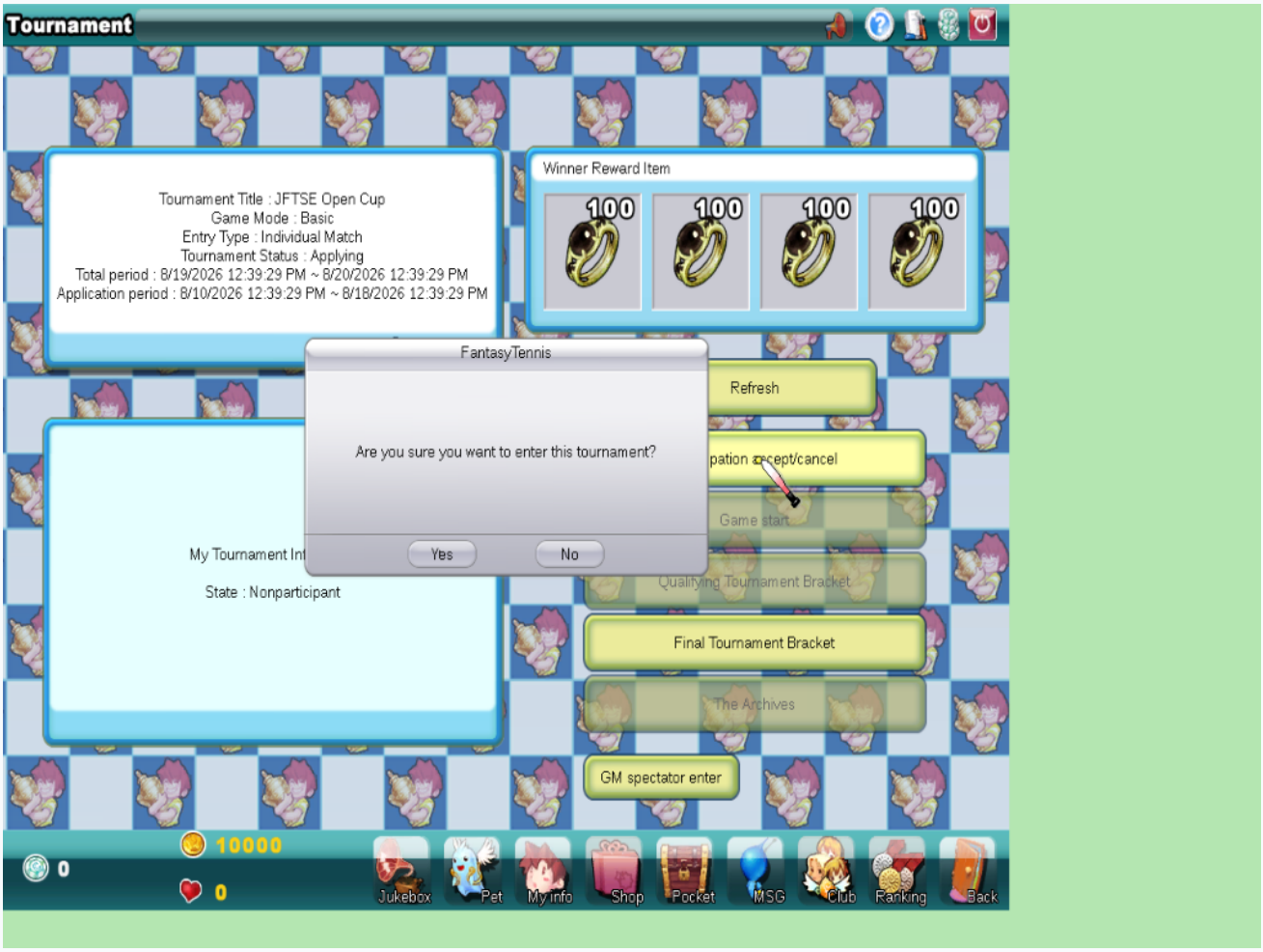


6. Tournament detail and winner rewards rendered

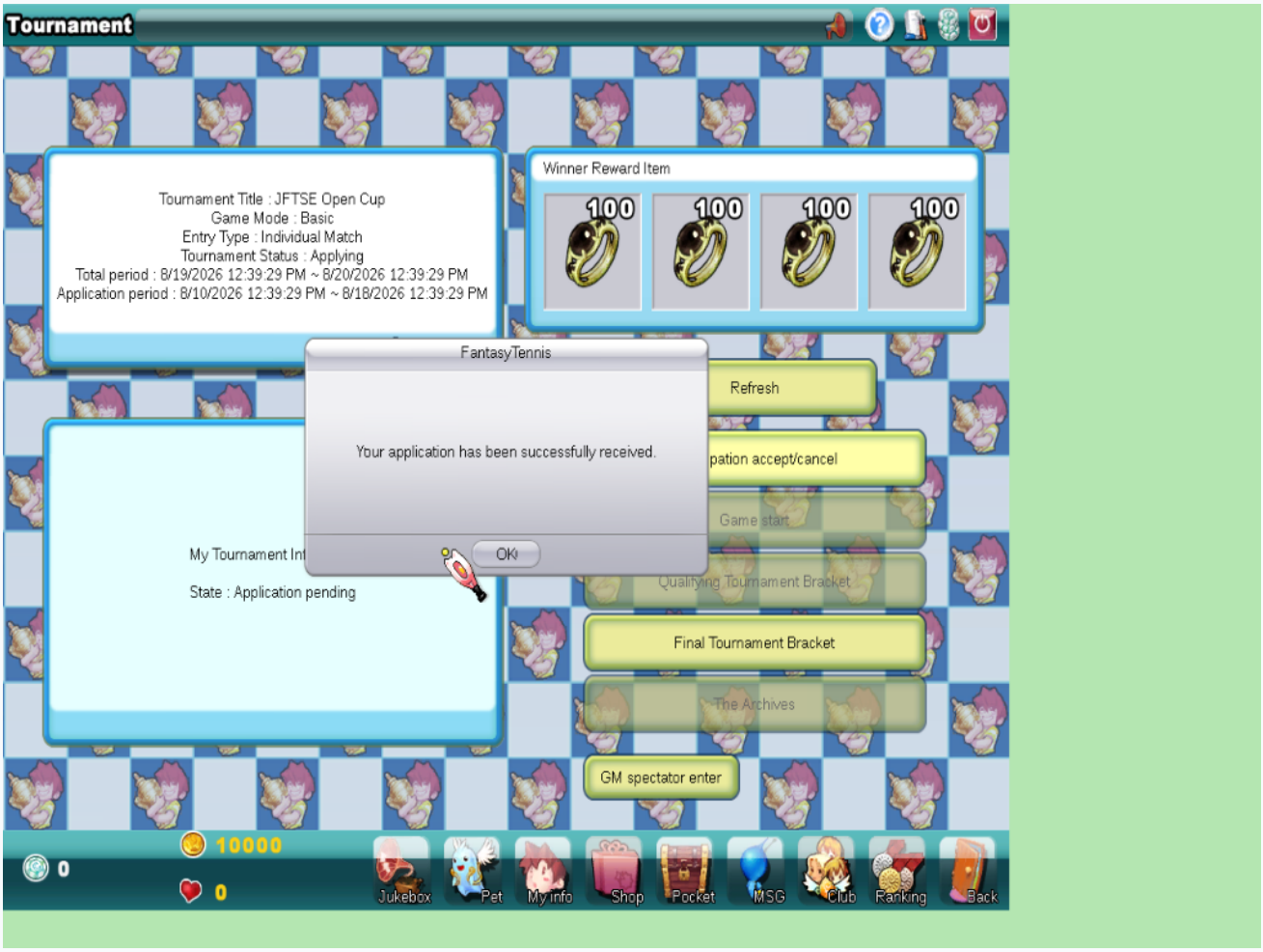
The client displays title, Basic game mode, Individual Match entry type, application dates, status, and winner reward items.



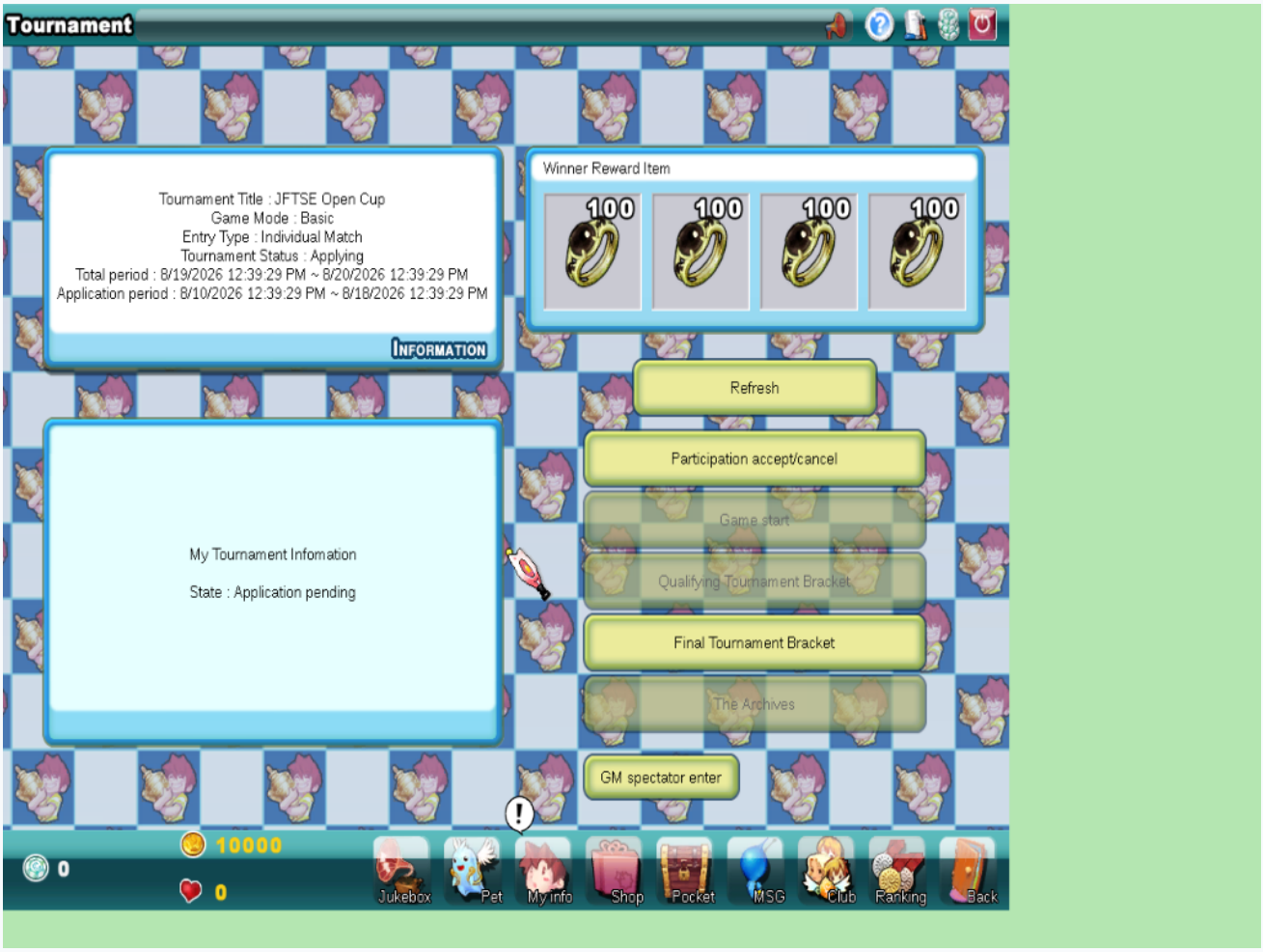
7. Apply confirmation



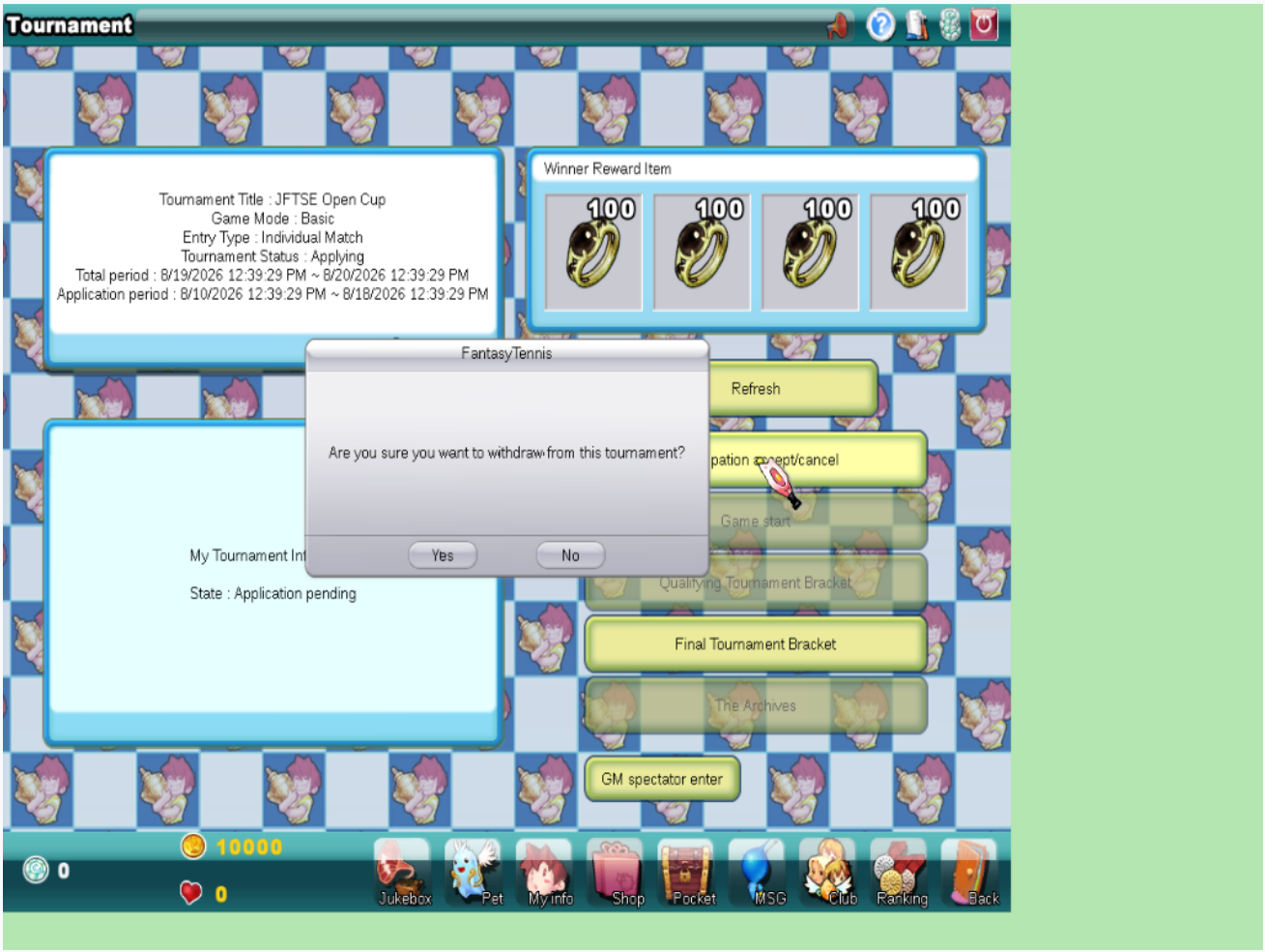
8. Apply success

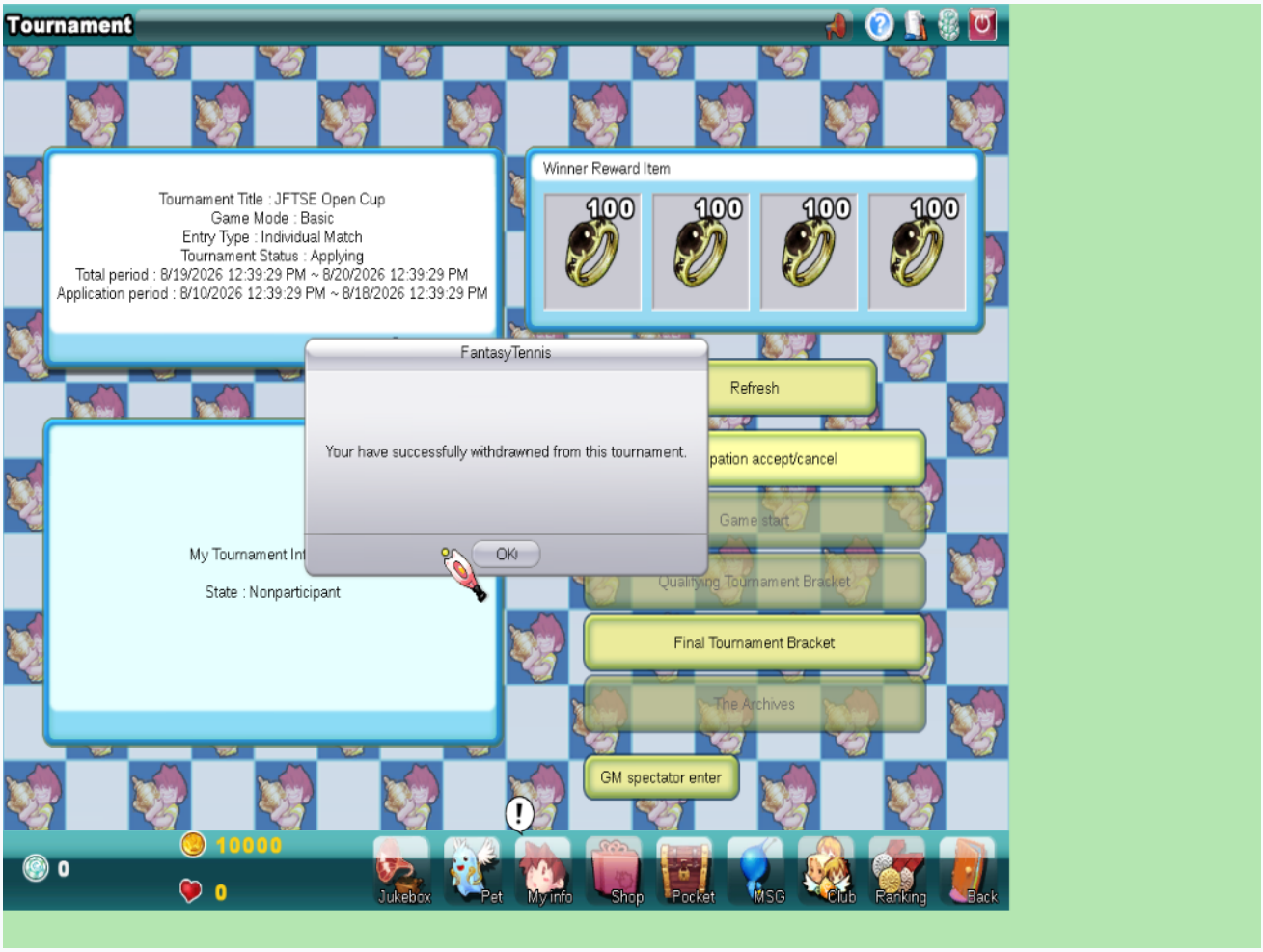


9. Applied state persisted across detail refreshes

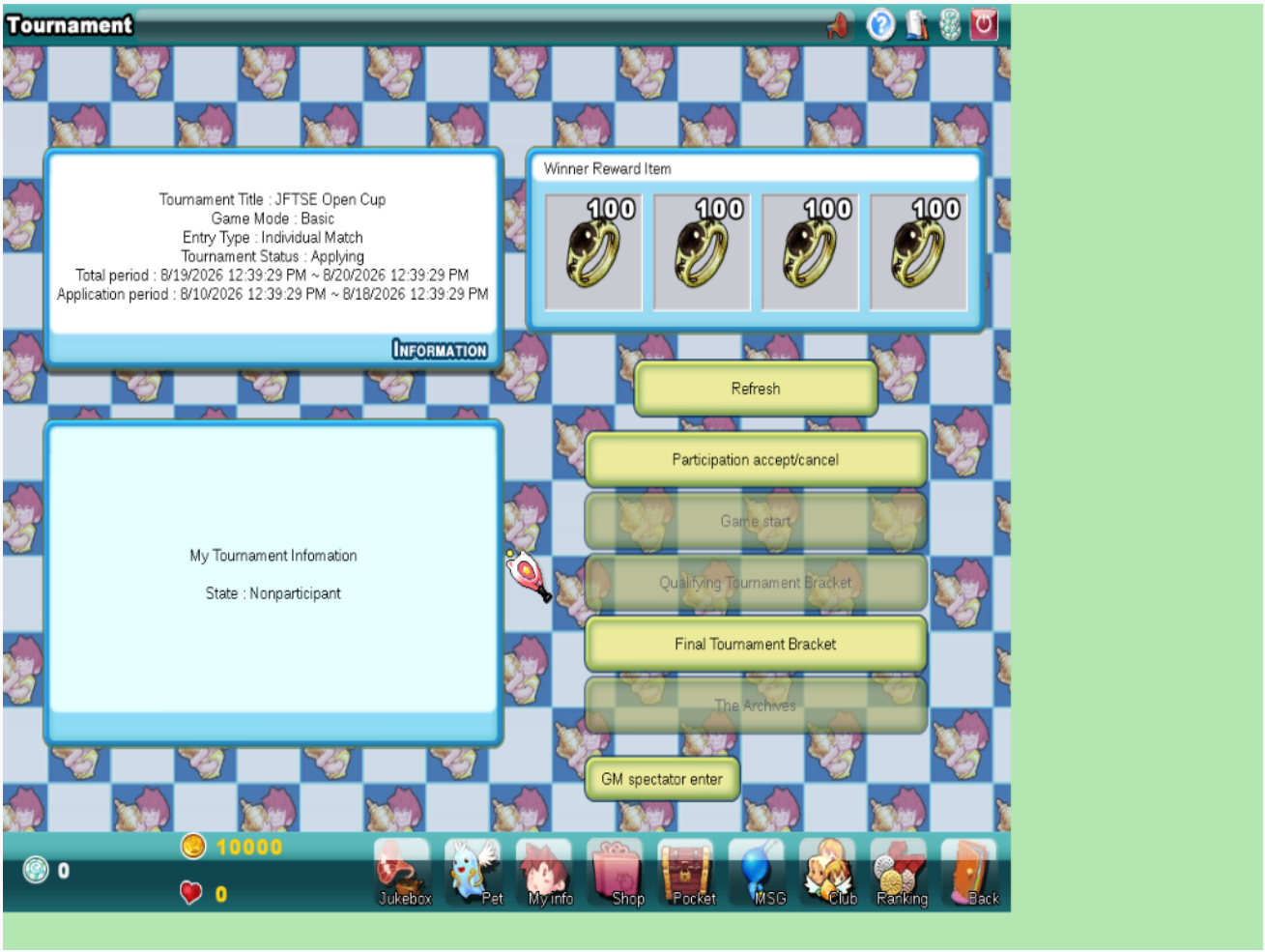


10. Cancel confirmation and success



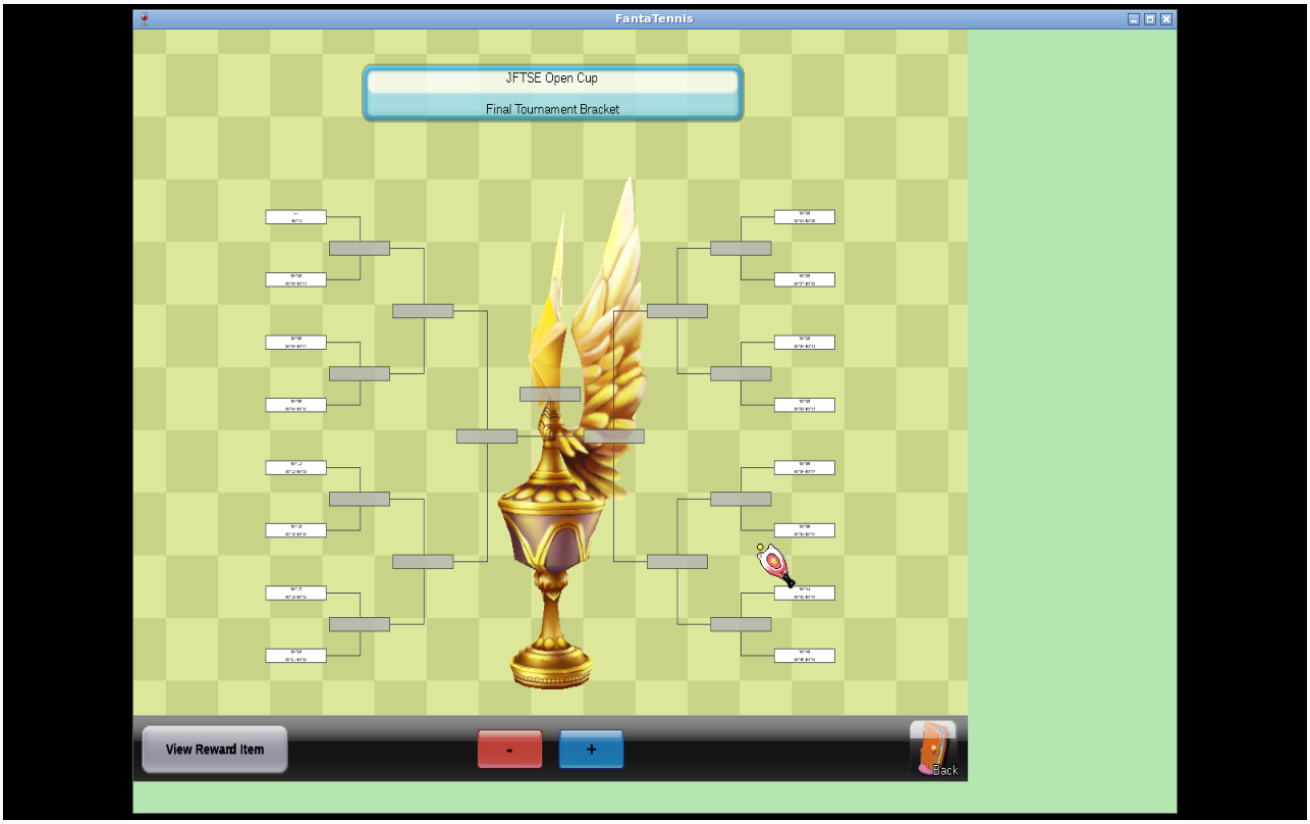

Tournament
Tournament Title : JFTSE Open Cup
Game Mode : Basic
Entry Type : Individual Match
Tournament Status : Applying
Total period : 8/19/2026 12:39:29 PM ~ 8/20/2026 12:39:29 PM
Application period : 8/10/2026 12:39:29 PM ~ 8/18/2026 12:39:29 PM
Winner Reward Item
100
100
100
100
FantasyTennis
Your have successfully withdrawned from this tournament.
OK
Refresh
pation accept/cancel
Game start
Qualifying Tournament Bracket
Final Tournament Bracket
The Archives
GM spectator enter
My Tournament Int
State : Nonparticipant
10000
0
0
Jukebox
Pet
My info
Shop
Pocket
MSG
Club
Ranking
Back

11. Nonparticipant state restored

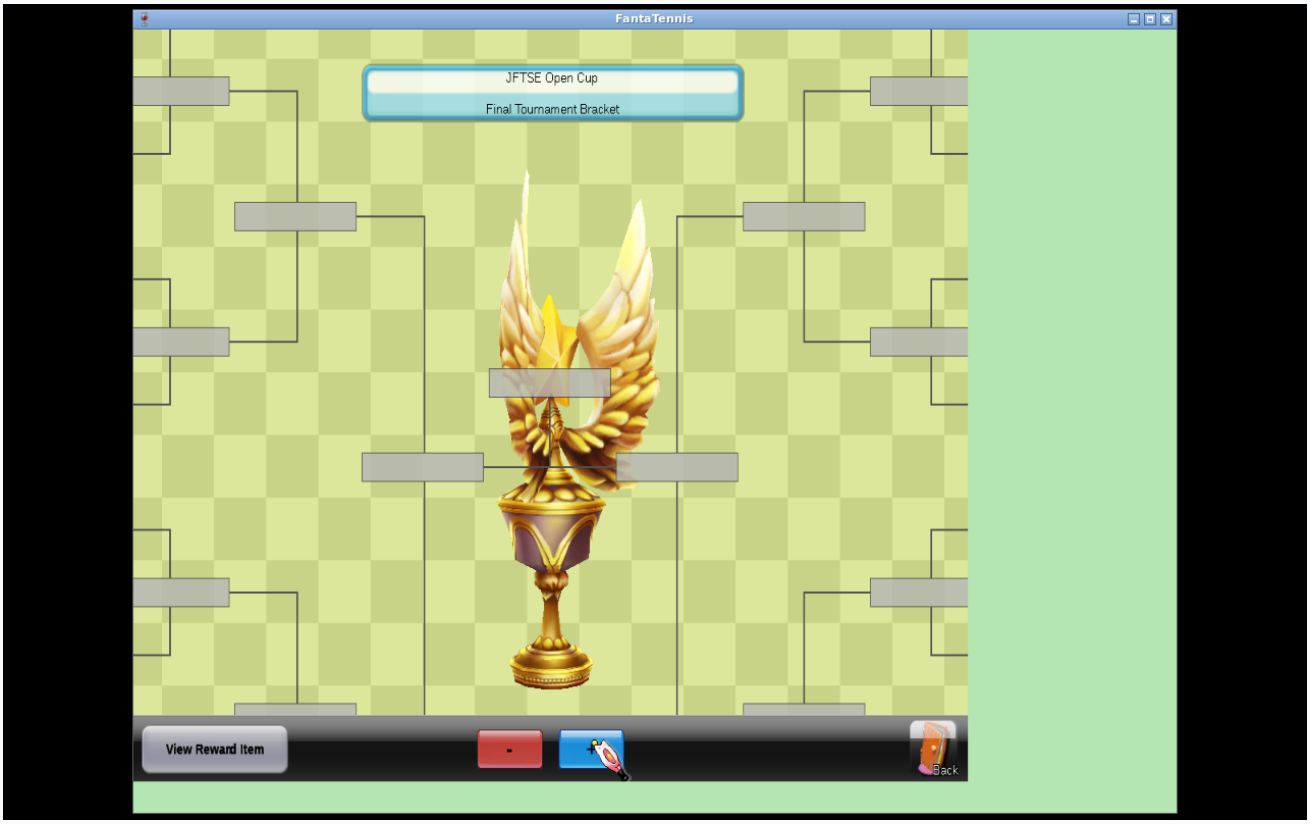


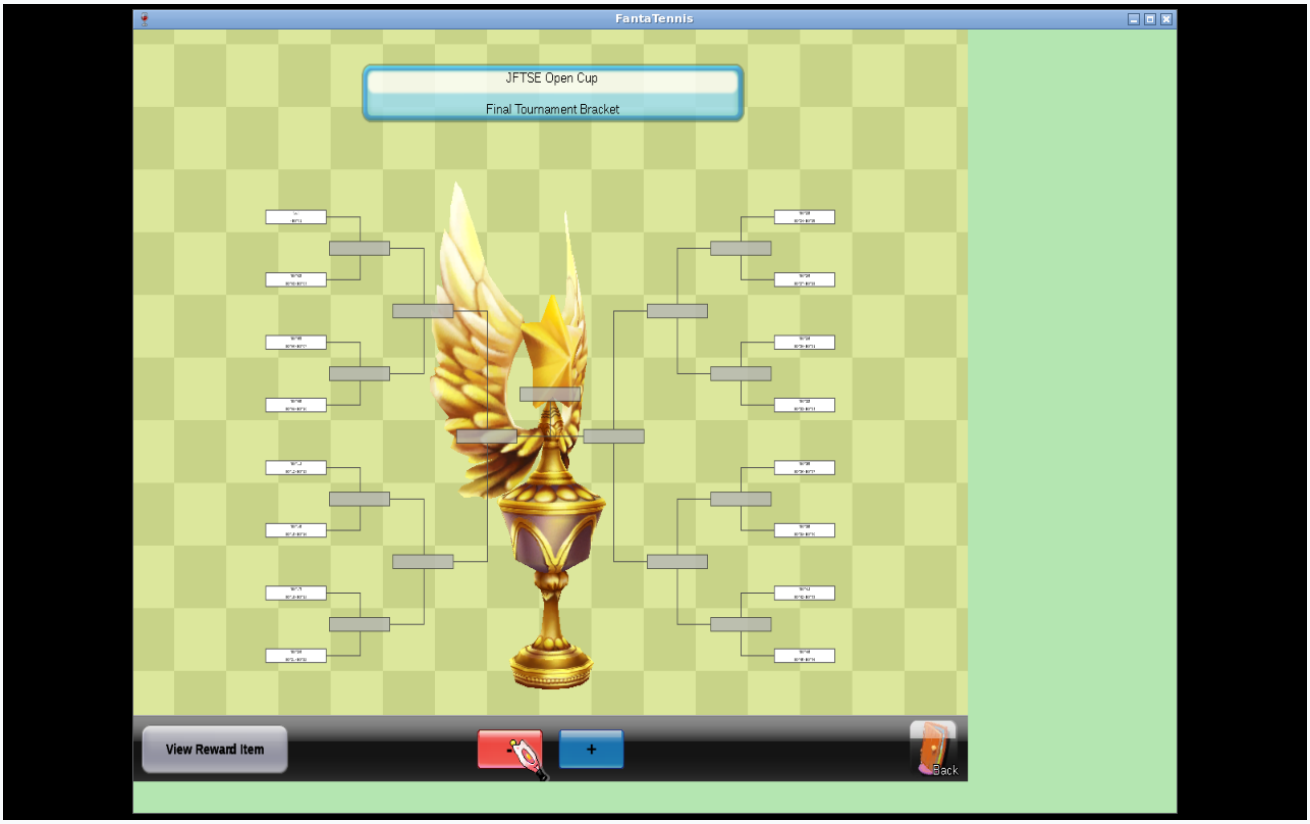
12. Final bracket visibly rendered

This is the decisive UI proof. It corresponds to the fresh PCAP containing `0x26C0`, `0x26C1`, automatic `0x26C2`, and accepted `0x26C3`, plus the runtime `c3_ok` trace.



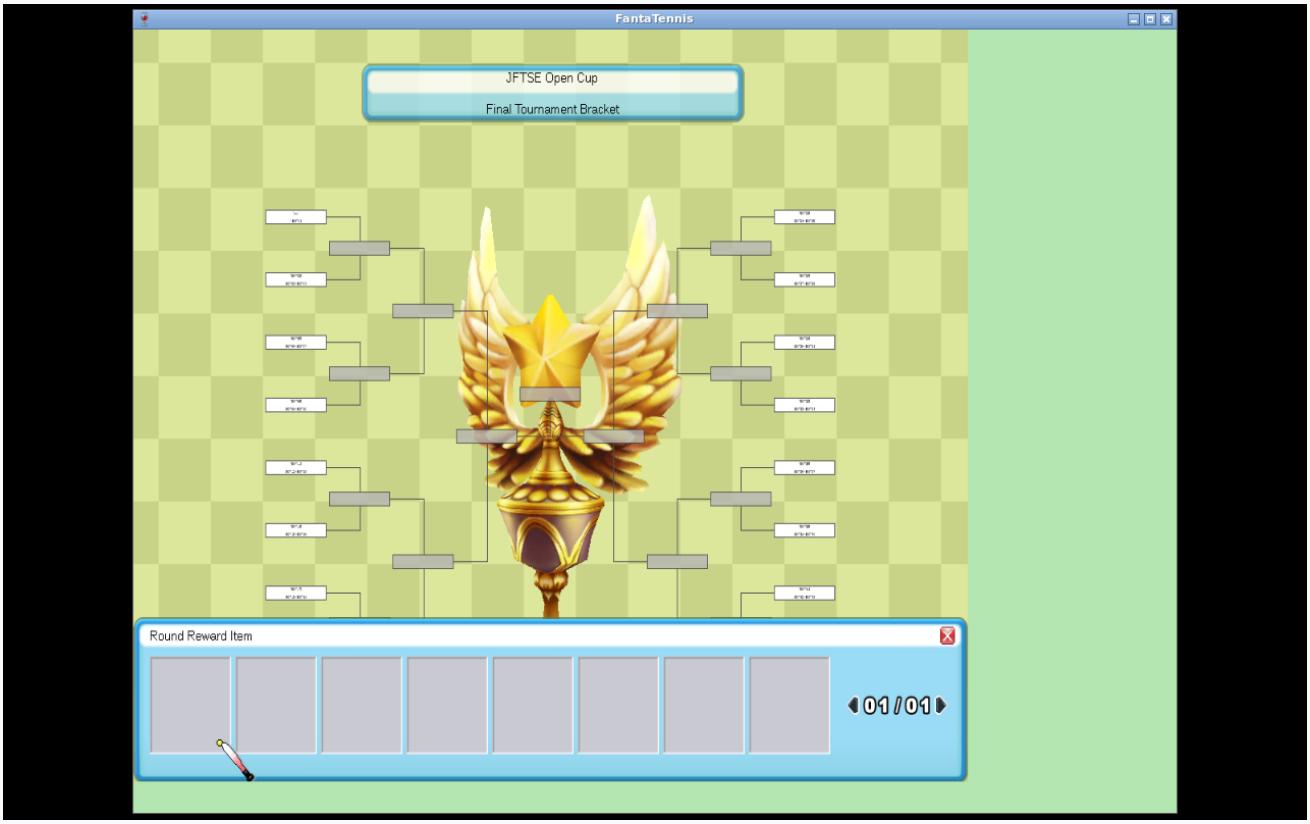
13. Zoom in and out controls worked

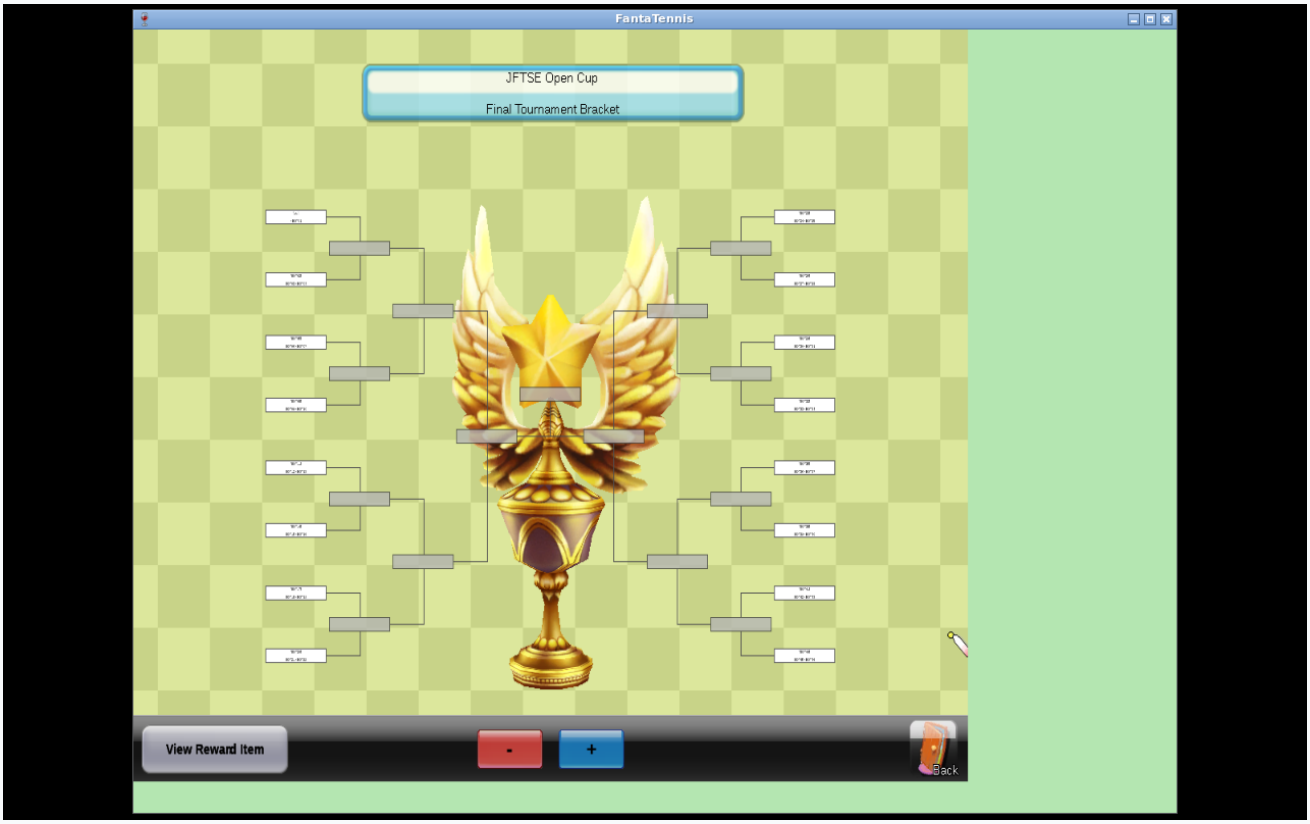




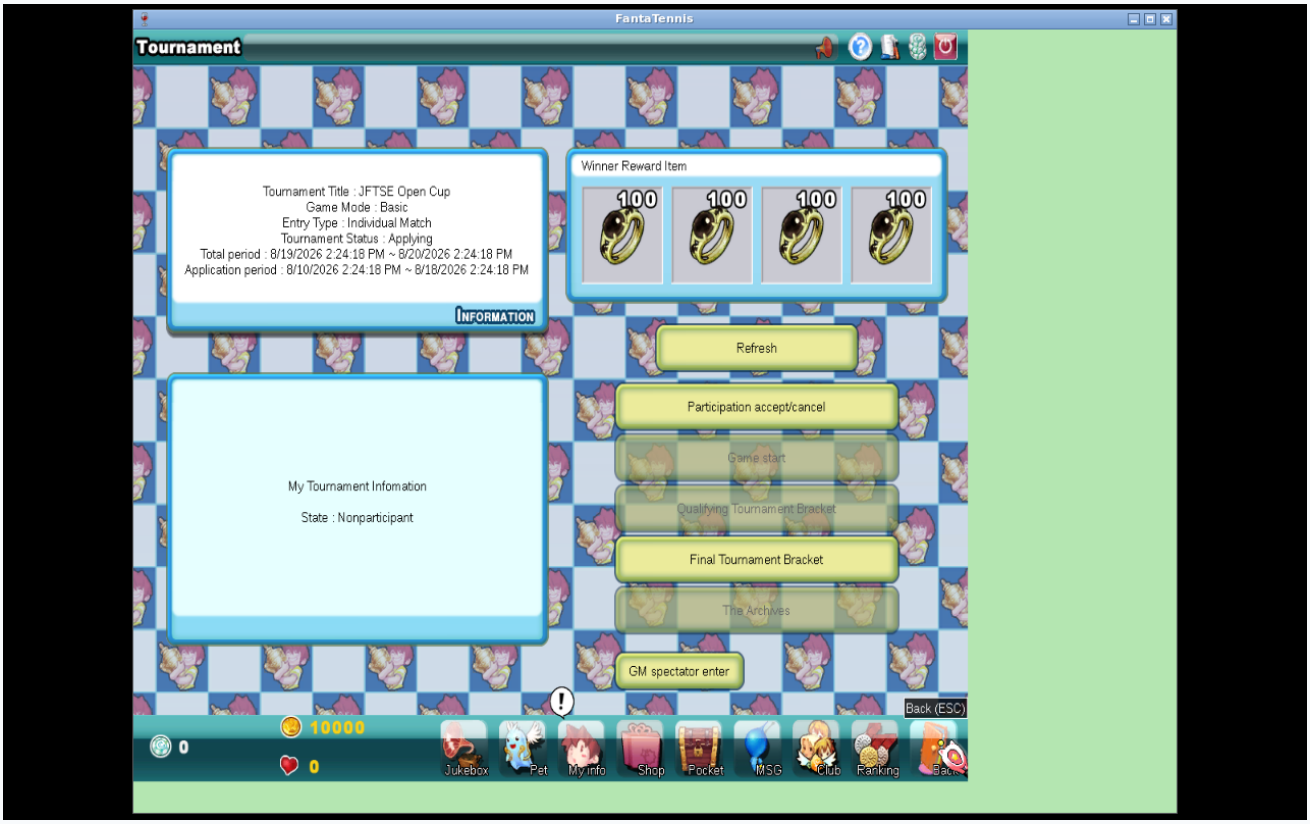
14. Round reward control opened and closed

The panel is empty because no round-progression rewards were invented. The already recovered winner rewards remain visible on the detail screen.





15. Back navigation returned to tournament details



Network and runtime evidence

Primary captures

Artifact	Purpose
`client-server-green.pcap`	Original list/detail/apply/cancel green session
`tournament-bracket-sentinel-green.pcap`	Fresh 328-frame final bracket session with accepted `FF 00` states
`tournament-complete-packet-exchange.tsv`	Human-readable tournament packet index extracted from the fresh capture
`tournament-bracket-sentinel-packets.tsv`	Exact first `C0/C1/C2/C3` exchange
`client-bracket-parser-success-trace.txt`	Count, comparison, and repeated client `c3_ok` probes
`game-server-bracket-sentinel-green.log`	Server-side decoded and encoded tournament traffic

First accepted bracket exchange

Frame	Direction	ID	Total TCP bytes	Key values
209	C→S	`0x26C0`	14	tournament 1, type 1, page 0
211	S→C	`0x26C1`	594	success, 16 fixed rows
212	C→S	`0x26C2`	14	tournament 1, type 1, index 0
213	S→C	`0x26C3`	47	success, count 15, `FF 00 × 15`

The client repeated `0x26C2` approximately every ten seconds. Every captured poll received the same 47-byte response and every instrumented parse took `c3_ok`.

Reproduction

Java validation

Use Java 21:

```
export JAVA_HOME=/tmp/jdk21
export PATH="$JAVA_HOME/bin:$PATH"

TOURNAMENT_TESTS=\
'TournamentBracketMatchPacketHandlerTest,'\
'TournamentBracketPacketHandlerTest,'\
'TournamentJoinPacketHandlerTest,'\
'TournamentManagerTest,'\
'TournamentPacketProtocolTest,'\
'TournamentInfoPacketHandlerTest'

mvn -pl game-server -am \
  -Dtest="$TOURNAMENT_TESTS" \
  -Dsurefire.failIfNoSpecifiedTests=false \
  test

mvn test
mvn -pl game-server -am -DskipTests package
```

PCAP extraction

```
sudo tshark \
  -r docs/tournament-mode/evidence/green/client-e2e/tournament-bracket-sentinel-green.pcap \
  -Y 'tcp.port == 5895 && tcp.len >= 8' \
  -T fields \
  -e frame.number -e frame.time_relative \
  -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
  -e tcp.len -e tcp.payload \
| grep -i -E 'ad26|ae26|af26|b026|b126|b226|b326|b426|be26|bf26|c026|c126|c226|c326'
```

Runtime client parser verification

The Wine client was run under Xvfb/Openbox. Linux tracefs uprobes were attached to the client executable at parser branches recovered from disassembly. The PE mapping used for this build was:

```
probe file offset = virtual address - 0x00400000
```

The decisive probes were:

```
0x5c9dcd  derived count/object state
0x5c9deb  received-versus-expected count comparison
0x5ca1c0  success return
0x5ca1cd  failure return
```

Probe offsets are build-specific and must not be reused against another executable hash.

Tests added

The test suite covers:

- decoding every recovered request field;
- exact packet IDs and little-endian payload order;
- success-only response tails;
- terminal one-byte failures;
- five reward records without a prefix;
- both fixed 16-pair arrays without prefixes;
- rejection of wrong fixed-array cardinality;
- full 16-row C1 serialization;
- exact 15-record `FF 00` C3 success payload;
- rejection of unrecovered bracket selectors;
- per-player apply/cancel isolation;
- unknown tournament rejection.

Known limitations and next evidence required

These are explicit evidence gaps, not hidden TODOs:

1. **Qualifying bracket:** capture and disassemble an enabled type-0 flow, including its cardinality and C1 row semantics.
2. **64-player relationship:** determine how the wiki's 64-player overall tournament maps to qualifying pages and the recovered 16-entry final stage.
3. **Lifecycle states:** recover list status values and date/state transitions that enable Game Start, qualifying bracket, and archives.
4. **Match entry:** recover `0x26B5–0x26B8` and any room/relay handoff without assuming they match ordinary lobby room creation.
5. **Spectator entry:** recover `0x26C6` and permissions before implementing the GM spectator control.
6. **Progression:** determine nonempty C3 result/state semantics from real completed matches; only the unresolved `FF 00` sentinel is proven.
7. **Round rewards:** recover the source and award transaction used by the round-reward panel.
8. **Persistence:** design entities only after the lifecycle and authoritative data source are known. The current wiki schema does not define tournament tables.
9. **Restart recovery:** persist registration, bracket, and settlement atomically once the database model is evidence-backed.
10. **Multi-client match E2E:** validate two or more real clients through tournament assignment, relay connection, result reporting, bracket advancement, and prize settlement.

Conclusion

The development branch previously exposed a Tournament entry point with no supporting server protocol. The recovered slice now drives the real client from login through listing, details, participation changes, and a visibly rendered final bracket. The strongest evidence is not a screenshot alone: the accepted PCAP, exact golden bytes, repeated runtime `c3_ok` traces, and full Java 21 reactor pass agree on the same contract.

The implementation remains intentionally conservative. It implements observed behavior and rejects unsupported selector variants. The remaining original-mode work is documented as a concrete evidence plan rather than filled with speculative packet fields or database tables.